

Property Location Analyzer

Sprawozdanie z projektu ASI

Piotr Wysocki, Miłosz Zieliński

Czerwiec 2026

Spis treści

1	Opis projektu	3
2	Architektura systemu	3
3	Moduły systemu	4
3.1	Data Downloader	4
3.2	Backend	4
3.3	Frontend	5
4	Baza danych	5
5	Testowanie	6
5.1	Testy jednostkowe i integracyjne	6
5.2	Testy wydajnościowe	7
6	Środowiska uruchomieniowe	8
7	Podsumowanie	9

1 Opis projektu

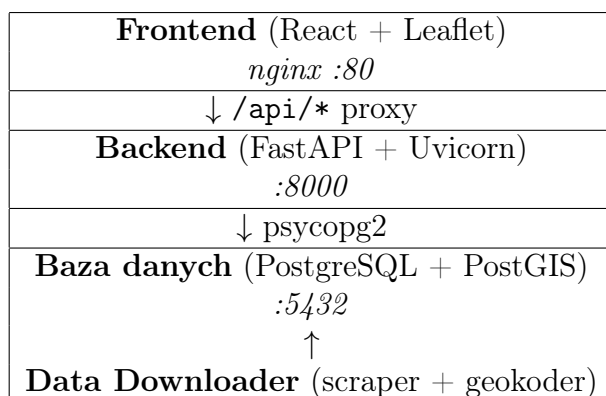
Nasz projekt z przedmiotu ASI (Architektura Systemów Informatycznych) to system służący do analizy ofert nieruchomości w Warszawie pod kątem ich lokalizacji względem punktów infrastruktury miejskiej (POI – *Points of Interest*). System pobiera ogłoszenia z portalu *adresowo.pl*, geokoduje je, a następnie kojarzy z pobliskimi obiektami takimi jak przystanki autobusowe, szkoły, restauracje czy szpitale. Użytkownik może przeglądać wyniki przez interfejs webowy: interaktywną mapę z możliwością filtrowania ofert według ceny, metrażu, liczby pokoi oraz odległości od wybranego rodzaju infrastruktury.

System jest w pełni skonteneryzowany przy użyciu **Docker Compose** i obsługuje cztery środowiska: deweloperskie (hot-reload), testowe (izolowana baza), produkcyjne oraz środowisko testów wydajnościowych (Locust).

Demo projektu można zobaczyć tutaj: [link](#)

2 Architektura systemu

System składa się z czterech głównych komponentów komunikujących się przez sieć Docker:



Rysunek 1: Uproszczony diagram architektury systemu

Warstwa	Technologia	Rola
Frontend	React 18, react-leaflet, Leaflet	Interfejs użytkownika
Backend	Python, FastAPI, Uvicorn	REST API
Scraper	Python, BeautifulSoup, Pandas	Pobieranie i przetwarzanie ofert
Baza danych	PostgreSQL 13+, PostGIS	Przechowywanie danych przestrzennych
Geokodowanie	Nominatim API (OpenStreetMap)	Konwersja adresów na współrzędne
POI	Overpass API (OpenStreetMap)	Pobieranie infrastruktury miejskiej
Konteneryzacja	Docker, Docker Compose	Izolacja i wdrożenie

Tabela 1: Stack technologiczny projektu

Diagram C4 można zobaczyć tutaj: [link](#)

3 Moduły systemu

3.1 Data Downloader

Moduł odpowiada za cały potok inżynierii danych. Składa się z pięciu plików:

- `scraper.py` – pobiera ogłoszenia z *adresowo.pl* przy użyciu `requests` i `BeautifulSoup`;
- `parser.py` – czyści i normalizuje dane (cena, metraż, liczba pokoi, adres);
- `geocoder.py` – konwertuje adresy na współrzędne geograficzne przez Nominatim API (opóźnienie 1,2 s między zapytaniami w celu zachowania limitu API), a także pobiera punkty POI z Overpass API;
- `database.py` – zapisuje oferty i POI do bazy PostgreSQL;
- `main.py` – orkiestruje potok i planuje jego cykliczne uruchamianie (nowe oferty codziennie o 02:00, odświeżenie POI w każdy poniedziałek o 03:00) przy użyciu biblioteki `schedule`.

Potok danych przebiega według następujących kroków:

1. **Scraping** – pobranie ≈ 40 ogłoszeń z portalu;
2. **Filtracja** – wybór ofert z wybranych dzielnic (Mokotów, Ochota, Rembertów);
3. **Geokodowanie** – przeliczenie adresów na pary (lat, lon);
4. **Zapis** – wstawienie wyników do tabeli `properties`.

Osobno uruchamiany jest potok POI, który pobiera z Overpass API 13 kategorii obiektów infrastruktury miejskiej dla całej Warszawy i zapisuje je do tabeli `points_of_interest`.

3.2 Backend

Backend to serwer REST API zbudowany na **FastAPI** z **Uvicorn**. Dostarcza trzy endpointy opisane w tabeli 2.

Metoda	Ścieżka	Opis
GET	/	Healthcheck
GET	/api/v1/properties	Lista nieruchomości z filtrami
GET	/api/v1/properties/{id}	Szczegóły oferty + 20 najbliższych POI
GET	/api/v1/ingestion/status	Statystyki bazy danych

Tabela 2: Endpointy REST API

Endpoint `/api/v1/properties` obsługuje siedem parametrów filtrowania:

Parametr	Typ	Opis
price_min / price_max	float	Zakres ceny (PLN)
area_min / area_max	float	Zakres metrażu (m ²)
rooms_min / rooms_max	int	Zakres liczby pokoi
price_per_sqm_max	float	Maksymalna cena za m ²
poi_category	string	Kategoria POI (np. bus_stop)
poi_radius_m	int	Promień szukania POI w metrach (50–5000)
limit	int	Maks. liczba wyników (1–1000, domyślnie 300)

Tabela 3: Parametry filtrowania endpointu `/api/v1/properties`

Filtr POI realizowany jest przez podzapytanie `EXISTS` z funkcją `ST_DWithin` z rozszerzenia PostGIS, co umożliwia efektywne wyszukiwanie przestrzenne w zasięgu podanego promienia.

Automatyczna dokumentacja Swagger UI dostępna jest pod adresem `/docs` po uruchomieniu serwera.

3.3 Frontend

Frontend to aplikacja jednostronicowa (SPA) napisana w **React 18**. Cała logika UI zawarta jest w jednym komponencie `App`, zarządzającym stanem przez hooki `useState` i `useEffect`.

Interfejs składa się z trzech obszarów:

- **Pasek nagłówka** – wyświetla liczbę ofert w bazie, liczbę punktów POI oraz datę ostatniego scrapingu (pobierane z `/api/v1/ingestion/status`);
- **Panel filtrów** (sidebar) – formularz z polami do filtrowania ofert; filtr POI wyświetla dodatkowy suwak promienia;
- **Mapa** – komponent `MapContainer` z `react-leaflet` z markerami dla każdej oferty; kliknięcie markera otwiera popup z podstawowymi danymi i przyciskiem *Infrastruktura*.

Po kliknięciu *Infrastruktura* aplikacja pobiera szczegóły oferty z `/api/v1/properties/{id}` i wyświetla modal z listą 20 najbliższych punktów POI posortowanych rosnąco po odległości.

W środowisku produkcyjnym aplikacja jest budowana do statycznych plików (`npm run build`), a **nginx** serwuje je na porcie 80. Żądania do `/api/*` są proxy-owane do backendu FastAPI.

4 Baza danych

Baza danych to **PostgreSQL** z rozszerzeniem **PostGIS** umożliwiającym przechowywanie i indeksowanie danych przestrzennych.

System korzysta z dwóch tabel:

Tabela properties

Kolumna	Typ	Opis
id	serial PK	Identyfikator
title	text	Tytuł ogłoszenia
address_clean	text	Znormalizowany adres
price	numeric	Cena (PLN)
area	numeric	Metraż (m ²)
rooms	integer	Liczba pokoi
geom	geometry(Point, 4326)	Współrzędne (PostGIS)
link	text UNIQUE	URL ogłoszenia
scraped_at	timestamp	Data pobrania

Tabela 4: Schemat tabeli properties

Tabela points_of_interest

Kolumna	Typ	Opis
id	serial PK	Identyfikator
category	text	Kategoria (np. bus_stop)
name	text	Nazwa obiektu
geom	geometry(Point, 4326)	Współrzędne (PostGIS)

Tabela 5: Schemat tabeli points_of_interest

Obsługiwane kategorie POI: bus_stop, school, hospital, cinema, theatre, museum, pub, restaurant, cafe, bank, parking, police, supermarket.

5 Testowanie

5.1 Testy jednostkowe i integracyjne

Testy napisano przy użyciu **pytest** i **httpx**. Podzielono je na trzy klasy:

TestQueryProperties (11 przypadków)

Testy weryfikują poprawność dynamicznego budowania zapytań SQL w funkcji `query_properties`. Sprawdzane scenariusze:

- brak filtrów – zapytanie zawiera tylko warunek bazowy (`geom IS NOT NULL`);
- filtry cenowe – poprawne dodanie warunków `price >= %s` i `price <= %s`;
- filtry metrażu i liczby pokoi;
- filtr ceny za m²;
- filtr POI – weryfikacja obecności podzapytania `EXISTS` z `ST_DWithin`;
- `limit` zawsze jako ostatni parametr;
- kombinacja wszystkich filtrów;
- wynik jako lista słowników;

- błąd połączenia z bazą zwraca HTTP 503.

TestQueryPropertyWithPois (5 przypadków)

Testy weryfikują funkcję pobierającą szczegóły pojedynczej nieruchomości wraz z listą pobliskich POI:

- zwrot nieruchomości z polem `nearest_pois`;
- nieistniejące ID zwraca HTTP 404;
- brak POI w pobliżu – puste `nearest_pois`;
- zapytanie o POI używa poprawnego `property_id`;
- błąd bazy – HTTP 503.

TestQueryIngestionStatus (4 przypadki)

Testy weryfikują endpoint statystyk:

- wynik zawiera wszystkie wymagane klucze;
- `poi_categories` jest słownikiem indeksowanym kategorią;
- wartości liczbowe zgodne z danymi z bazy;
- błąd bazy – HTTP 503.

Baza danych jest mockowana przez `unittest.mock.patch`, co pozwala uruchamiać testy bez rzeczywistego połączenia z PostgreSQL. Środowisko testowe (Docker) używa oddzielnej bazy `realestate_test`.

5.2 Testy wydajnościowe

Testy wydajnościowe zrealizowano przy użyciu **Locust**. Scenariusz symuluje realistyczny ruch: użytkownik dwa razy częściej klika w szczegóły oferty niż korzysta z filtrów. Przetestowano dwa rodzaje filtrów:

- **filtr lekki** – filtrowanie po cenie (proste zapytanie po polach numerycznych);
- **filtr ciężki** – filtrowanie po kategorii i promieniu POI (złączenia przestrzenne z `ST_DWithin`).

Wyniki testów wydajnościowych

Scenariusz / Zapytanie	Requests	Fails	Mediana	Średnia
<i>Scenariusz A: 10 użytkowników, ramp-up 2</i>				
Szczegóły oferty (/properties/1)	~196	0	<10 ms	17 ms
Filtr lekki (cena)	~62	0	28 ms	28 ms
Filtr ciężki (POI)	~52	0	860 ms	876 ms
<i>Scenariusz B: 50 użytkowników, ramp-up 10</i>				
Szczegóły oferty	1042	0	420 ms	435 ms
Filtr lekki	563	0	560 ms	537 ms
Filtr ciężki	523	0	1200 ms	1180 ms
<i>Scenariusz C: 100 użytkowników, ramp-up 50</i>				
Szczegóły oferty	537	0	2200 ms	2169 ms
Filtr lekki	240	0	860 ms	839 ms
Filtr ciężki	238	0	4900 ms	4871 ms

Tabela 6: Wyniki testów wydajnościowych Locust

Wnioski z testów wydajnościowych

1. **Stabilność (brak błędów)** – we wszystkich scenariuszach liczba błędów wyniosła 0. System pod dużym obciążeniem nie odrzuca połączeń, lecz kolejkuje zapytania i odpowiada z opóźnieniem.
2. **Wąskie gardło – złączenia przestrzenne** – filtr POI jest najdroższą operacją w systemie. Przy 100 równoczesnych użytkownikach średni czas odpowiedzi dla tego endpointu wynosi $\approx 4,9$ s. Operacje GIS (ST_DWithin na kolumnie `geography`) są obliczeniowo kosztowne.
3. **Liniowa degradacja wydajności** – wraz ze wzrostem ruchu z 10 do 100 użytkowników czasy odpowiedzi rosną liniowo dla wszystkich zapytań, co wskazuje na prawidłowe zarządzanie połączeniami bez katastroficznego załamania systemu.

6 Środowiska uruchomieniowe

System obsługuje cztery środowiska Docker Compose:

Środowisko	Plik compose	Cechy
Deweloperskie	<code>docker-compose.dev.yml</code>	Hot-reload, porty otwarte na hoście
Testowe	<code>docker-compose.test.yml</code>	Izolowana baza <code>realestate_test</code> , pytest
Produkcyjne	<code>docker-compose.prod.yml</code>	Build statyczny, nginx, bez mount'ów
Wydajnościowe	<code>docker-compose.perf.yml</code>	Locust UI na porcie 8089

Tabela 7: Środowiska uruchomieniowe

Baza danych (`docker-compose.yml`) jest wspólnym plikiem bazowym dla wszystkich środowisk. Schemat bazy inicjowany jest skrypcem `database/init.sql` przy pierwszym uruchomieniu kontenera.

7 Podsumowanie

Projekt ASIProjekt zrealizowano jako kompletny, wielowarstwowy system webowy do analizy nieruchomości. Zaimplementowano:

- automatyczny potok danych (scraping, geokodowanie, pobieranie POI) z cyklicznym harmonogramem;
- REST API z filtrowaniem przestrzennym opartym na PostGIS;
- interaktywny interfejs mapowy w React;
- pełną konteneryzację w czterech środowiskach;
- 20 testów jednostkowych z mockowaniem bazy danych;
- testy wydajnościowe w trzech scenariuszach obciążenia.

System wykazał się stabilnością – brak błędów przy obciążeniu do 100 równoczesnych użytkowników. Zidentyfikowano wąskie gardło w postaci zapytań geoprzestrzennych (`ST_DWithin`), które przy dużym ruchu mogą wymagać optymalizacji przez indeksy przestrzenne GiST lub materializację wyników dla popularnych promieni.